

Compound Risk Monitor (CRM) - Complete Workflow Documentation

Documentation Date: September 15, 2026 Reflects: the `databricks` branch of `github.com/compoundrisk/monitor` (the branch Databricks builds from every day) as of the date above.

Read This First: Databricks Code vs. Data Freshness

If a file looks "off" or out of date on Databricks, read this before assuming something is broken.

On Databricks, `working_path` and `mounted_path` point at two different, unrelated storage locations:

- `working_path` = `/tmp/crm/monitor` — ephemeral cluster storage. This is where the code that actually runs lives: every job run re-clones (or fetches/merges) the `databricks` branch of `monitor` + `hosted-data` into `/tmp` before anything executes. `/tmp` is wiped on cluster restart, and it **cannot be seen at all** in the Databricks file browser or the Unity Catalog Volumes UI.
- `mounted_path` = `/dbfs/mnt/CompoundRiskMonitor` — the persistent DBFS mount (also browsable via Unity Catalog Volumes, at `/Volumes/prd_datascience_compoundriskmonitor/volumes/compoundriskmonitor`). This is where **outputs** land (`output/...` , `production/crm-dashboard-prod.csv`) and it does get refreshed by every successful run — but it holds almost none of the actual source code.

In practice:

- `src/*.R` files looking old or different from GitHub in that Databricks/Volumes browser is **expected, not a bug** — you're looking at the persistent DBFS mount, not the ephemeral `/tmp` clone that actually executed.
- Output/data files** (anything under `output/...` or `production/...`) looking stale is a real signal worth investigating — something didn't run, or didn't write where expected.

Full technical explanation, including a related `run_type` gotcha this same split caused, is in [General Architecture](#) below.

Table of Contents

- [1. Read This First: Databricks Code vs. Data Freshness](#)
- [2. Executive Summary](#)
- [3. General Architecture](#)
- [4. 2-Phase Workflow](#)
- [5. Local Environment vs Databricks](#)
- [6. Data Structure](#)
- [7. How to Generate Dashboard Inputs](#)
- [8. ACAPS Manual Review Process](#)
- [9. Technical Details](#)
- [10. Manual Maintenance Checklist](#)
- [11. Known Issues](#)

Executive Summary

The **Compound Risk Monitor (CRM)** is a comprehensive global crisis risk monitoring system that integrates data from **6 risk dimensions** from multiple external sources. The system is designed to run in both a **local environment** and on **Databricks**, automatically detecting where it is running.

Main Function: Collect raw data from APIs and manual uploads → Normalize it (0-10) → Aggregate by country and dimension → Produce a final dataset (`crm-dashboard-data.csv`) that feeds the interactive dashboard at <https://compoundrisk.worldbank.org>

Execution Time: ~10-15 minutes locally (depending on API availability and how many sources need a fresh download)

Three repositories work together:

- `monitor` (this repo) — the R code and orchestration notebooks
- `hosted-data` — manually-uploaded source files, ACAPS review files, and (for a local run) all generated outputs
- `monitor-xlsx` — the public Excel workbook, published separately

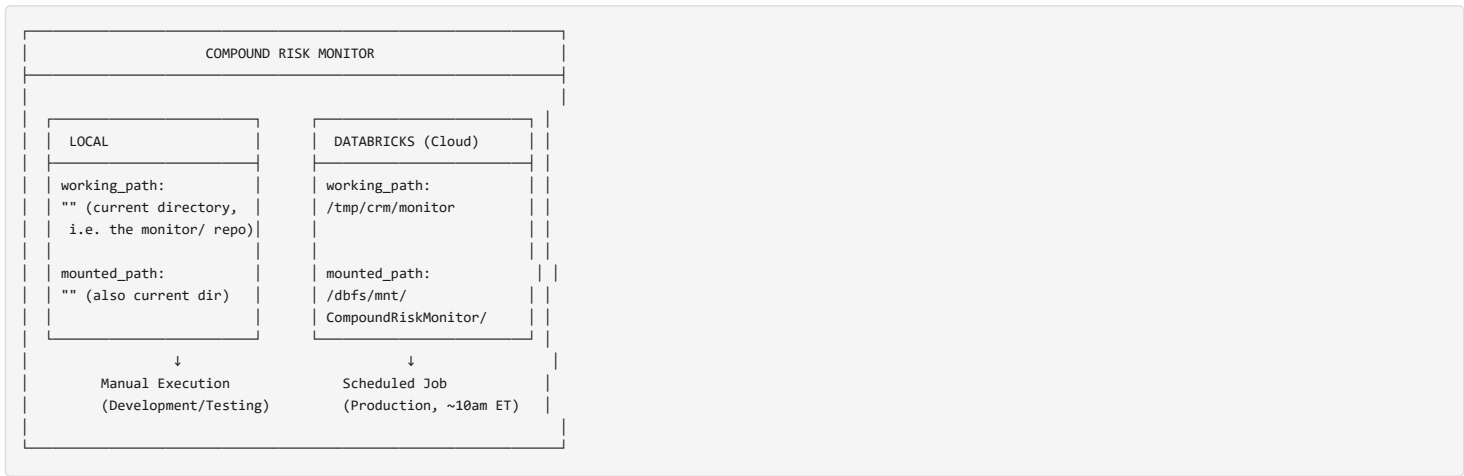
For a local run, `hosted-data` and `monitor-xlsx` must be cloned inside `monitor/` , as siblings of `src/` .

 **Before you go looking for a bug because a file "looks old" on Databricks, read the [callout in General Architecture](#) — the code and the data live on two different filesystems there, and only one of them is what the Databricks file browser shows you.**

General Architecture

Dual Environment Structure

The system implements an **environment-agnostic model** that adapts automatically based on a single check (`dir.exists("/dbfs")`):



🚨 **Correction vs earlier drafts of this document:** locally, `mounted_path` and `working_path` are empty strings, not "hosted-data". That means every output path built with `paste_path(mounted_path, "output/...")` resolves to a plain `output/` folder — a sibling of `hosted-data/`, directly under `monitor/`. `output/` is not created inside `hosted-data/`. This matches both the code (`01-update-inputs.R` / `02-process-indicators.R`, lines 14-23) and the repo layout Ben described when onboarding the team.

🚨 IMPORTANT: what you see in the Databricks file browser can be stale — here's why

This trips people up regularly, so it's worth being explicit: on Databricks, `working_path` and `mounted_path` point at two different kinds of storage, and only one of them holds live code.

- `working_path = /tmp/crm/monitor` — this is where the code actually runs from. At the start of every job, `run-notebooks.R` calls `src/clone-git-repos-into-tmp.sh`, which either clones `monitor` + `hosted-data` fresh into `/tmp/crm` (if they're not there yet) or does a `git fetch origin databricks && git merge origin/databricks` on what's already there. Either way, by the time `source('src/db-notebooks/01-update-inputs.R')` runs, the code in `/tmp` is guaranteed to match the tip of the `databricks` branch. But `/tmp` is **ephemeral cluster storage** — it's wiped on cluster restart, and it is a completely different filesystem from what the Databricks UI's file browser shows you.
- `mounted_path = /dbfs/mnt/CompoundRiskMonitor` — this is the **persistent** DBFS mount. It's also browsable through the newer Unity Catalog Volumes UI at `/Volumes/prd_datascience_compoundriskmonitor/volumes/compoundriskmonitor` (same underlying storage, just a different UI on top of it). This is where `output/` (all archived inputs, dimension sheets, `crm-dashboard-data.csv`, `crm-excel/`, etc.) and `production/crm-dashboard-prod.csv` actually land, and those files genuinely do get refreshed by every successful run. But almost none of the **source code** lives here — the one exception is `clone-git-repos-into-tmp.sh` itself, which gets copied back to `/dbfs/.../src/` after every clone/update so the bootstrap script is always findable even before `/tmp` has anything in it.

What this means in practice:

- If you're looking at **data/output files** (anything under `output/` or `production/`) in the Volumes/DBFS browser and they look stale, that's a real signal — something didn't run or didn't write where you expect. Worth investigating (see the `run_type` gotcha immediately below for one concrete way this bites people).
- If you're looking at **`src/*.R` code files** in that same browser and they look old or different from what's on GitHub, that's **expected and not a bug** — the code that actually executed lives in the ephemeral `/tmp` clone, which the Databricks file browser can't see at all.

Related gotcha, now fixed (2026-09-16): `run_type` (which decides whether outputs land in `output/scheduled/` vs `output/manual/`) is read via `dbutils.widgets.get("run_type")`, but no notebook in this repo ever *declares* that widget with `dbutils.widgets.text(...)` — it only ever gets a value if the Databricks Job's task passes it in as a parameter. Since the "New Daily job" task didn't have that parameter configured, `.get()` always threw and silently fell back to `"manual"` — **every single run, including scheduled ones** — so `output/scheduled/` (and `output/scheduled/crm-excel/`, etc.) sat stale for a long time while fresh data was actually landing in `output/manual/` instead. Fixed by adding a `run_type = scheduled` parameter to the job's task in the Databricks Workflows UI (Job → task → Edit task → Parameters). If `output/scheduled/` ever looks stale again, check that parameter first before assuming the pipeline itself is broken.

Key System Files

```
monitor/
├─ src/
│  ├─ db-notebooks/          ← Main scripts
│  │  ├─ 01-update-inputs.R  (Collects raw data)
│  │  ├─ 02-process-indicators.R (Processes and aggregates)
│  │  └─ run-notebooks.R      (Runs both notebooks in sequence)
│  ├─ fns/                  ← Auxiliary functions
│  │  ├─ prep.R              (Setup and library loading)
│  │  ├─ indicators.R        (~3,140 lines – one collect()/process() pair per source)
│  │  ├─ aggregation.R       (Dimension aggregation, labels)
│  │  └─ helpers.R           (paste_path, archiveInputs, delay_error, etc.)
│  ├─ country-groups.csv     (Country/region reference, auto-refreshed)
│  └─ indicators-list.csv     (Active-indicator metadata, actively used by the pipeline)
├─ output/                  ← Created on first run; NOT inside hosted-data/
│  ├─ inputs-archive/        (Raw archived data, one CSV/folder per source)
│  ├─ manual/                (Outputs from a local/manual run)
│  ├─ scheduled/             (Outputs from the Databricks daily job)
│  ├─ production/            (crm-dashboard-prod.csv – file the dashboard reads)
│  └─ errors.log              (Written by delay_error() when errors are captured)
├─ hosted-data/             ← Separate repo, cloned as a subfolder
│  ├─ <source>/              (Manually-uploaded editions, e.g. eiu/, fsi/, gfsi/)
│  ├─ acaps-*temp-auto.csv    (Generated by every run; needs manual review)
│  ├─ acaps-risk-list-reviewed/ (Reviewed ACAPS files, by dimension)
│  └─ .access/                (Credential files – not committed, created locally/on Databricks)
├─ monitor-xlsx/            ← Separate repo, public Excel workbook
└─ init-script/
   ├─ compoundriskmonitor.sh  (older cluster-start script; superseded, see below)
   └─ compoundriskmonitor_v2.sh (actual cluster-start script in use as of this doc)
```

2-Phase Workflow

Phase 1: Raw Data Collection (01-update-inputs.R)

Duration: ~5-10 minutes

This script calls one `*_collect()` function per source and archives the result incrementally via `archiveInputs()`. As of this writing, the calls are grouped like this:

Dimensions and Collection Calls (Phase 1)

Dimension	Calls in 01-update-inputs.R	Notes
HEALTH	<code>ghsi_collect()</code> , <code>dons_collect()</code> , <code>ifrc_collect()</code>	GHSI reads a manually-uploaded <code>hosted-data/ghsi/ghsi.csv</code>
FOOD SECURITY	<code>fpi_collect_api()</code> , <code>proteus_collect()</code> , <code>gfsi_collect()</code>	<code>fews_collect_api()</code> and <code>fao_wfp_web_collect()</code> are commented out — see Known Issues
MACRO FISCAL	<code>eiw_collect_many()</code> , <code>mfr_watchlist_collect()</code>	<code>mfr_watchlist_collect()</code> runs but its output is not consumed in Phase 2 (see below)
SOCIO-ECONOMIC	<code>mpo_collect()</code> , <code>mfr_collect()</code> , <code>imf_collect()</code>	<code>mfr_collect()</code> reads <code>hosted-data/efi-mfr/macrofin.csv</code> (household/macro-financial risk) — confusingly named, distinct from <code>mfr_watchlist_collect()</code> above
NATURAL HAZARDS	<code>gdacs_collect_many()</code> , <code>inform_risk_collect()</code> , <code>fao_locust_multi_collect()</code> , <code>inform_severity_collect()</code> , <code>acaps_risk_list_collect()</code>	<code>iri_collect()</code> is commented out ("Pause IRI") but IRI is still <i>processed</i> in Phase 2 from whatever was last archived
FRAGILITY & CONFLICT	<code>fsi_collect()</code> , <code>fcs_collect()</code> , <code>acled_collect()</code> , <code>gic_collect()</code> , <code>ifes_collect()</code>	<code>reign_collect()</code> is commented out; GIC + IFES are combined downstream into a "pseudo-REIGN" indicator (see Phase 2)

Each collector is wrapped in `delay_error(return = NA, on = error_delay)`. **error_delay only equals TRUE when the Databricks job widget error_delay is set** — in a plain local run it defaults to `FALSE`, which means `delay_error()` does **not** catch errors: if one source throws (bad credentials, a changed API, a stale cookie), the whole script stops right there. On Databricks the daily job is expected to set the widget so the run continues past a single failing source and reports it at the end via `release_delayed_errors()`.

Archiving Mechanism

`archiveInputs()`:

1. Reads new data from the just-collected source
2. Reads previous data from the matching CSV/folder in `output/inputs-archive/`
3. Combines both, de-duplicating by the given grouping key (usually `Country`, sometimes `Country + date`)
4. Writes the combined result back, so the archive grows over time instead of being overwritten

```
archiveInputs(new_data,
              group_by = "Country",  # de-duplication key
              today = file_date)     # date stamp for this edition, when relevant
```

`output/inputs-archive/` contains one entry per source, for example: `ghsi.csv`, `who_dons.csv`, `gfsi.csv`, `wb_fpi.csv` (Food Price Inflation), `eiw.csv`, `mpo-alt.csv`, `inform_risk.csv`, `macrofin.csv`, `mfr_watchlist.csv`, `fao_locust.csv`, `acaps_risklist.csv` (Natural Hazard keyword matches), `fsi.csv`, `fcs.csv`, `acled.csv`, `gic.csv`, `ifes.csv`, plus subfolders for sources with dated editions: `fews/`, `inform-severity/`, `fao-wfp-web/`.

Phase 2: Processing and Aggregation (02-process-indicators.R)

Duration: ~2-5 minutes

- 1. Reads archived data from Phase 1 (via each source's *_process() function)
- 2. Normalizes and derives indicator scores
- 3. Aggregates each dimension with aggregate_dimension()
- 4. Combines all six dimensions into one wide table
- 5. Reshapes to the long dashboard format and writes every output artifact

Step 2.1: ACAPS "temp-auto" files (runs first, before the dimension sheets)

```
write.csv(acaps_risk_list_process(as_of, dim = "Socioeconomic", prefix = "S_"),
          "hosted-data/acaps-socio-temp-auto.csv", row.names = F)
write.csv(acaps_risk_list_process(as_of, dim = "Natural Hazard", prefix = "NH_"),
          "hosted-data/acaps-natural-temp-auto.csv", row.names = F)
write.csv(acaps_risk_list_process(as_of, dim = "Conflict and Fragility", prefix = "Fr_"),
          "hosted-data/acaps-conflict-temp-auto.csv", row.names = F)
```

These three files land at the root of hosted-data/ (not inside output/) and are the basis for the ACAPS Manual Review Process.

Step 2.2: Per-dimension aggregation

Each dimension is built with aggregate_dimension("<Dimension Name>", <indicator_1_process()>, <indicator_2_process()>, ...) . The actual current composition per dimension is:

Dimension	*_process() calls actually used	Written to
Health	ghsi_process() , dons_process() , ifrc_process()	output/.../dimensions/health-sheet.csv
Food Security	gfsi_process() , proteus_process() , fews_process() , fpi_process() , fao_wfp_web_process()	.../food-sheet.csv
Macro Fiscal	eiw_process() only — mfr_process() is commented out	.../macro-sheet.csv
Socioeconomic Vulnerability	inform_socio_process() , mpo_process() , macrofin_process() , imf_process() , acaps_risk_list_reviewed_process(dim = "Socioeconomic")	.../socio-sheet.csv
Natural Hazard	gdacs_process() , inform_nathaz_process() , iri_process() , fao_locust_process() , inform_severity_process() , acaps_risk_list_reviewed_process(dim = "Natural Hazard")	.../natural_hazards-sheet.csv
Conflict and Fragility	fsi_process() , fcs_process() , acled_process() , acled_events_process() , eiw_security_process() , acaps_risk_list_reviewed_process(dim = "Conflict and Fragility") , pseudo_reign_process()	.../fragility-sheet.csv

Notes on the differences from earlier drafts of this doc:

- Food Security now includes Hunger Hotspots (fao_wfp_web_process()) — see Known Issues for why it's currently running on stale data.
- Macro Fiscal is EIU-only today. mfr_watchlist_collect() still runs in Phase 1, but nothing in Phase 2 reads it — it is effectively dead weight in the current pipeline.
- Socioeconomic pulls in INFORM's socio-economic sub-index and the Socioeconomic-dimension ACAPS review, neither of which appeared in earlier versions of this table.
- GIC and IFES are not separate indicators. pseudo_reign_process() combines them into a single "coup / irregular election" flag, standing in for the paused REIGN dataset.
- acaps_risk_list_reviewed_process() (used here) is distinct from acaps_risk_list_process() (used in Step 2.1) — the former reads the human-reviewed files, the latter is the raw keyword-matched output.

A *_process() function typically:

- Normalizes values to a 0-10 scale (10 = maximum risk), using normfuncpos() / normfuncneg()
- Assigns labels (Low / Medium / High)
- Returns one row per country, ready to be merged with the other indicators in the dimension

Step 2.3: Combining Dimensions

```
all_dimensions <- list(
  health_sheet, food_sheet, macro_sheet,
  socio_sheet, natural_hazards_sheet, fragility_sheet
)%>%
  reduce(full_join, by = "Country") %>%
  count_flags(outlook = "emerging", high = 10, medium = 7) %>%
  count_flags(outlook = "underlying", high = 10, medium = 7) %>%
  count_flags(outlook = "overall", high = 7, medium = 5) %>%
  mutate(Countryname = iso2name(Country), .after = Country)

multi_write.csv(all_dimensions, "crm-wide.csv", c(output_directory, archive_directory))
```

Note the label thresholds differ by outlook: Underlying/Emerging indicators use High ≥ 10 (they're often discrete 0/3/5/7/10 scores), while the Overall risk score uses High ≥ 7, Medium 5-6.9, Low < 5.

Step 2.4: Transformation to Dashboard Format ★

```

long <- pretty_col_names(all_dimensions) %>%
  lengthen_data() %>%
  add_secondary_columns(as_of) %>%
  round_value_col() %>%
  factorize_columns() %>%
  order_columns_and_rows() %>%
  mutate(Index = create_index(rename(., Indicator = Key)), .before = 1)

dashboard_data <- subset(long, `Data Level` != "Reliability") %>% add_overall_indicators()
dashboard_crisis <- label_crisis(dashboard_data) %>% mutate(Countryname = tolatin(Countryname))

multi_write.csv(dashboard_crisis, "crm-dashboard-data.csv", c(output_directory, archive_directory))
write.csv(dashboard_crisis, paste_path(mounted_path, "production/crm-dashboard-prod.csv"), row.names = F)

```

Key columns in `crm-dashboard-data.csv`: Country, Countryname, Indicator, Key, Value, Value_Label, Data_Level (Raw Indicator Data / Reliability / Dimension / Aggregated), Outlook (Underlying / Emerging / Overall), Source.

The **production** file the dashboard actually reads is `production/crm-dashboard-prod.csv` under `mounted_path` — i.e. `output/production/crm-dashboard-prod.csv` locally, `/dbfs/mnt/CompoundRiskMonitor/production/crm-dashboard-prod.csv` on Databricks. There is no `production(creadamanual)/` folder in the codebase — if you have one locally, it's a personal/local artifact, not something the pipeline creates.

Step 2.5: Public Excel and indicator dates

```

write_excel_source_files(
  all_dimensions = all_dimensions,
  health_sheet = add_dimension_dates(health_sheet, "Health", dimension_highs),
  # ... one sheet per dimension ...
  directory_path = paste_path(output_directory, "crm-excel/"))

ind_list <- date_indicators()
write.csv(ind_list, paste_path(output_directory, "crm-excel", "indicators-list-dated.csv"), row.names = F, na = "")

```

This step does NOT produce `crm.xlsx` itself — that's a correction vs earlier drafts of this doc. `write_excel_source_files()` (in `src/fns/aggregation.R`) only writes the per-dimension CSV source files into `crm-excel/`:

- `health.csv`
- `food-security.csv`
- `macro-fiscal.csv`
- `socioeconomic-vulnerability.csv`
- `natural-hazard.csv`
- `conflict-and-fragility.csv`
- `reliability-sheet.csv`
- `indicators-list-dated.csv`

On Databricks (with `run_type` correctly set to `scheduled`, see the callout above), these land at `/Volumes/prd_datascience_compoundriskmonitor/volumes/compoundriskmonitor/output/scheduled/crm-excel/`.

Turning these into `crm.xlsx` is a separate, manual process — nothing in this repo automates it. `crm.xlsx` lives in the `monitor-xlsx` repo (<https://github.com/compoundrisk/monitor-xlsx/raw/main/crm.xlsx>) and is what's served by the "Data download" link on the public dashboard landing page, <https://compoundrisk.worldbank.org/app-landing>. The exact manual steps for refreshing `crm.xlsx` from the `crm-excel/` CSVs above aren't documented anywhere in this repo — whoever owns that step needs to be asked directly. See also [Known Issues](#) for the current state of the dashboard this workbook feeds.

Step 2.6: Archiving this run

```

output/<run_type>/archive/run_<YYYY-MM-DD>/
├─ dimensions/
├─ crm-wide.csv
├─ crm-dashboard-data.csv
└─ crm-excel/

```

`run_type` is `manual` locally or `scheduled` when triggered by the Databricks job widget.

Local Environment vs Databricks

Automatic Detection

```
if (dir.exists("/dbfs")) {
  mounted_path <- "/dbfs/mnt/CompoundRiskMonitor"
  working_path <- "/tmp/crm/monitor"
  setwd(working_path)
} else {
  mounted_path <- ""
  working_path <- ""
}

inputs_archive_path <- paste_path(mounted_path, "output/inputs-archive/")
run_type <- tryCatch(dbutils.widgets.get("run_type"), error = function(e) "manual")
output_directory <- paste_path(mounted_path, "output/", run_type)
```

Environment Comparison

Aspect	LOCAL	DATABRICKS
working_path / mounted_path	"" (current directory)	/tmp/crm/monitor / /dbfs/mnt/CompoundRiskMonitor
Where output/ lands	monitor/output/ (sibling of hosted-data/)	/dbfs/mnt/CompoundRiskMonitor/output/
run_type default	manual	scheduled (set by the job widget)
Execution	source() a script, or Rscript ...	Databricks Workflow daily-run-job
Error handling	Any single source failure stops the whole run (error_delay defaults FALSE)	Continues past failures if the error_delay widget is set TRUE ; failures are collected and raised together at the end
Credentials	.access/ folder at the top of the monitor repo (not committed)	Same folder, provisioned separately on the mount
Main Use	Development, debugging, generating the ACAPS review files	Production — the dashboard's data comes from here

Data Flow

```
LOCAL (optional, mainly for ACAPS review / debugging)
  01-update-inputs.R → 02-process-indicators.R
  → output/manual/crm-dashboard-data.csv

hosted-data/ manual uploads and reviewed ACAPS files
  → pushed to the `databricks` branch of the hosted-data repo

DATABRICKS (production, daily job ~10am ET)
  clones monitor (databricks branch) + hosted-data + monitor-xlsx
  → runs run-notebooks.R
  → writes output/scheduled/crm-dashboard-data.csv
  → writes /dbfs/mnt/CompoundRiskMonitor/production/crm-dashboard-prod.csv
  → dashboard (compoundrisk.worldbank.org) reads that file
```

 Data Structure

Folder Tree (as created by the pipeline itself)

```

monitor/                                # this repo, `databricks` branch
├─ src/
│   ├─ fns/
│   │   ├─ prep.R
│   │   ├─ indicators.R                # ~3,140 lines
│   │   ├─ aggregation.R
│   │   └─ helpers.R
│   ├─ db-notebooks/
│   │   ├─ 01-update-inputs.R
│   │   ├─ 02-process-indicators.R
│   │   └─ run-notebooks.R
│   ├─ country-groups.csv
│   ├─ country-numbers.csv
│   ├─ countrylist.csv
│   ├─ factor-orders.csv
│   ├─ region-names.csv
│   └─ indicators-list.csv             # actively used metadata (indicators-schedule.csv, if present, is a legacy 2021-2022 reference and is NOT read by the code)
├─ output/                             # created on first run, sibling of hosted-data/
│   ├─ inputs-archive/                 # raw archived data, one file/folder per source
│   │   └─ manual/                    # local-run outputs
│   │       ├─ dimensions/            # 6 risk sheets
│   │       ├─ crm-wide.csv
│   │       ├─ crm-dashboard-data.csv  ★ key file
│   │       ├─ crm-excel/
│   │       ├─ runs/
│   │       └─ dimension-highs.csv
│   │           └─ archive/run_<date>/
│   ├─ scheduled/                     # Databricks daily-job outputs (same layout)
│   └─ errors.log
├─ hosted-data/                       # separate repo, cloned inside monitor/
│   ├─ .access/                       # credentials, not committed
│   ├─ acaps-*temp-auto.csv
│   ├─ acaps-risk-list-reviewed/
│   ├─ eiu/, fsi/, gfsi/, ghsi/, fcs/, imf-unemployment/, inform-risk/,
│   │   └─ food-price-inflation/, efi-mfr/, proteus/, gic/                # manually-uploaded editions
├─ monitor-xlsx/                      # separate repo, public workbook
└─ init-script/
    ├─ compoundriskmonitor.sh          # older version, superseded
    └─ compoundriskmonitor_v2.sh       # actual cluster-start script, see Known Issues

```

`production/crm-dashboard-prod.csv` lives under `mounted_path`, i.e. `output/production/` locally or `/dbfs/mnt/CompoundRiskMonitor/production/` on Databricks — not in a folder called `production(creadamanual)/`.

How to Generate Dashboard Inputs

Prerequisites

1. R with the packages loaded by `src/fns/prep.R` (`dplyr`, `tidyr`, `readr`, `stringr`, `purrr`, `countrycode`, `sf`, `raster`, `exactextractr`, `httr`, `httr2`, `rvest`, `jsonlite`, `xml2`, `lubridate`, `zoo`, `wppExplorer`, ...)
2. `hosted-data` and `monitor-xlsx` cloned as subfolders of `monitor/`
3. A `.access/` folder at the top of `monitor/` containing:
 - `acaps-credentials.csv` (columns: `username,password`)
 - `acled.csv` (columns: `username,key`)
 - `ifes-authorization.txt` (one line: the authorization token)
 - `iri-access.txt` (one line, the `__d1auth_id` cookie value)

Step 1: Data Collection

```
source("src/db-notebooks/01-update-inputs.R")
```

Downloads/reads all sources for which a `*_collect()` call is active and archives them in `output/inputs-archive/`. Any single failed source stops the script locally (see [Environment Comparison](#)) — re-run after fixing the failing source, `archiveInputs()` is idempotent so it's safe to re-run.

Step 2: Processing

```
source("src/db-notebooks/02-process-indicators.R")
```

Produces the six dimension sheets, `crm-wide.csv`, `crm-dashboard-data.csv`, the Excel workbook, and (importantly) the three ACAPS `*-temp-auto.csv` files that need human review before the next production run.

Verify success:

```
dir output\inputs-archive\  
dir output\manual\  
Test-Path output\manual\crm-dashboard-data.csv
```

Alternative: Run Both Phases

```
source("src/db-notebooks/run-notebooks.R")
```

Production

The daily Databricks job writes directly to `/dbfs/mnt/CompoundRiskMonitor/production/crm-dashboard-prod.csv`, which the dashboard reads. A local run does **not** publish to production by itself — it writes to your local `output/production/`. To affect production you either (a) push code/hosted-data changes to the `databricks` branch so tomorrow's scheduled job picks them up, or (b) run the notebooks interactively on the Databricks cluster.

ACAPS Manual Review Process

Why Manual Review is Needed

ACAPS Risk List events are matched to a dimension by **keyword matching**, which is not fully accurate: keywords can be negated in the text, or apply to only some of the listed countries even though an event covers several. A human review step decides which matched events actually count.

The three files to review (regenerated on every `02-process-indicators.R` run)

- `hosted-data/acaps-conflict-temp-auto.csv`
- `hosted-data/acaps-natural-temp-auto.csv`
- `hosted-data/acaps-socio-temp-auto.csv`

Each has 8 columns: `Countryname`, `iso3`, `risk_level_auto` (the original auto-assigned 0/3/7/10 level), `risk_level` (**editable** — starts equal to `risk_level_auto`), `risk_text` (short excerpt), `risk_text_full` (full ACAPS text), `last_risk_update`, `Review` (**editable**, `TRUE` / `FALSE`).

Note from the process README (`hosted-data/acaps-risk-list-reviewed/README.md`): the temp-auto CSVs only contain events updated **within the past two months and after the most recently reviewed file** — so each review only needs to cover new/changed events, not the full backlog. Also, one ACAPS event can list multiple countries; the CSV groups by event rather than by country.

How to Review

- Open each temp-auto CSV in Excel; resize/wrap cells to read `risk_text_full` comfortably.
 - For every row: set `Review = TRUE` if the event is a real risk for that country, `FALSE` otherwise.
 - If the assigned severity is wrong, edit `risk_level` (0-10) — `risk_level_auto` is left untouched as a record of what the algorithm originally picked.
 - Save the reviewed file into the matching folder, with the review date in the filename:
 - `hosted-data/acaps-risk-list-reviewed/Conflict and Fragility/acaps-conflict-YYYY-MM-DD.csv`
 - `hosted-data/acaps-risk-list-reviewed/Natural Hazard/acaps-natural-YYYY-MM-DD.csv`
 - `hosted-data/acaps-risk-list-reviewed/Socioeconomic/acaps-socio-YYYY-MM-DD.csv`
- Push `hosted-data` to its `databricks` branch.

How Reviews Are Used

```
acaps_risk_list_reviewed_process(dim = "Conflict and Fragility", prefix = "Fr_", as_of = as_of)
```

This reads **the most recent reviewed file** for the dimension, keeps only rows with `Review = TRUE`, and uses the (possibly edited) `risk_level`. For dates before `2023-04-19` (before this review process existed) the code falls back to the unreviewed, auto-matched events.

Current Status

Based on the files actually present in `hosted-data/acaps-risk-list-reviewed/` (all three dimensions), **the most recent completed review is dated 2025-07-11**. That is well over a year old as of this documentation date — confirm the current state directly in the `hosted-data` repo before relying on this number, but treat ACAPS as a source that is likely overdue for review.

Technical Details

Normalization Functions

- Positive normalization** (higher raw value = higher risk), e.g. inflation:

```
r normfuncpos(df, uperrisk, lowerrisk, "col") # value >= uperrisk -> 10; value <= lowerrisk -> 0; linear in between ``
```
- Negative normalization** (lower raw value = higher risk), e.g. Health Security Index:

```
r normfuncneg(df, uperrisk, lowerrisk, "col") # value <= uperrisk -> 10; value >= lowerrisk -> 0; linear in between ``
```

Dimension Aggregation

```
aggregate_dimension(dim, ..., prefix = "", overall_method = "geometric",
                    indicators_list_file = NULL)
```

Merges the supplied `*_process()` outputs, computes an overall score per country (geometric mean of the component indicators by default — Conflict and Fragility historically used an arithmetic mean instead, see the comment in `02-process-indicators.R`), and labels it with `assign_ternary_labels()`. Label thresholds are passed per call, not hardcoded: **Overall** uses High ≥ 7 / Medium 5-6.9 / Low < 5 ; **Underlying** and **Emerging** flag counts use High ≥ 10 / Medium ≥ 7 .

Error Handling

```
delay_error <- function(expr, return = NULL, no_stop = F, on = T, file.path = ...) {
  if (on) {
    tryCatch(expr, error = function(e) {
      # records to `delayed_error` and to output/errors.log, then returns `return`
    })
  } else {
    expr # <- on = FALSE: no try/catch at all, a real error stops the script
  }
}
release_delayed_errors() # called at the end of 02-process-indicators.R; re-raises any captured errors
```

Every call in the notebooks passes `on = error_delay`. **error_delay** is **FALSE** by default (it's read from a Databricks widget that doesn't exist locally), so a local run has **no** error tolerance unless you explicitly set `error_delay <- TRUE` before sourcing the notebooks.

Timing Functions

```
lap_start()
ghsi_collect(); dons_collect()
lap_print("Health dimension finished collecting") # prints elapsed time since lap_start()
```

Manual Maintenance Checklist

Cross-referenced against Ben's onboarding emails and the sources actually read from `hosted-data/` in the current code.

A. Still manual — upload a new edition when one is released

Status notes below reflect what was actually on disk in a local `hosted-data` checkout, cross-checked against each publisher's site, as of this documentation date. Re-verify before treating them as current.

Source	Folder	Where to get it	Cadence	Status (as observed)
GHSI	<code>hosted-data/ghsi/ghsi.csv</code>	ghsindex.org — download data model, copy overall score	Annual	Publisher's last edition is 2021 — no newer edition has been released since. Our copy is up to date.
GFSI	<code>hosted-data/gfsi/</code>	impacteconomist.com/sustainability/project/food-security-index/	Annual	Publisher's last edition is 2022 (11th iteration) — the index appears paused/discontinued since. Our copy (<code>gfsi-2022-09-01.csv</code>) is up to date.
EIU Operational Risk	<code>hosted-data/eiu/</code>	viewpoint.eiu.com/analysis/risk/operational-risk — set frequency to Monthly, download	Monthly (watch the off-by-one-month bug handled in <code>eiu_process()</code>)	Newest file on disk: <code>EIU_OR_Tracker_ByGeography-2026-09-15</code> (2026-09-15) — up to date.
FSI (Fragile States Index)	<code>hosted-data/fsi/</code>	fragilestatesindex.org/global-data/	Annual	Publisher's last available edition is 2024 . Our copy (<code>fsi-2024-11-20.xlsx</code>) is up to date.
FCS (Fragile & Conflict-affected)	<code>hosted-data/fcs/</code>	worldbank.org FCS classification page — use <code>fcs_create_file()</code> helper	Annual	Up to date — reflects the official FY2027 list and new methodology, published at https://www.worldbank.org/en/brief/2026/07/01/classification-of-fragile-and-conflict-affected-situations .
IMF Unemployment	<code>hosted-data/imf-unemployment/</code>	IMF World Economic Outlook Database, Unemployment Rate, ISO3 + <code>.</code> decimal	Semi-annual (WEO publishes ~April and ~October)	Newest file on disk: <code>imf-unemployment-2026-04-01.csv</code> — up to date with the April 2026 WEO edition. The October 2026 edition will be the next one to watch for.
INFORM Risk	<code>hosted-data/inform-risk/</code>	drmkc.jrc.ec.europa.eu/inform-index — use the conversion script in the folder	Semi-annual (publisher updates ~March and ~September)	Newest file on disk: <code>inform-risk-2026-08-31.csv</code> — up to date with the 2026 edition.
EFI/MFR household & macro-financial risk	<code>hosted-data/efi-mfr/macrofin.csv</code>	Internal WB Macro-Financial Review (Finance/EFI practice), published on an internal SharePoint site (<code>worldbankgroup.sharepoint.com/sites/wbfinance/.../macrofinancial-reviews</code>) — not publicly accessible, requires the WB Finance team's file	Quarterly	Single undated file on disk, no version history — freshness can't be assessed from the filename; confirm with the source owner. Note: only two of the file's ~10 risk categories (Household risks, overall Macro-Financial Risk) are pulled into the CRM, both landing in the Socioeconomic Vulnerability sheet despite the <code>M_</code> prefix on the latter.

Source	Folder	Where to get it	Cadence	Status (as observed)
Proteus Composite Index	hosted-data/proteus/proteus.csv	WFP-developed food security composite index; see "The Proteus composite index" and https://dataviz.vam.wfp.org/economic/food-security-index	Ad hoc (no fixed publication schedule)	Publisher's last edition is 2017 — no newer edition has been released since. Our copy is up to date. Feeds the Food Security dimension as <code>F_Proteus_Score</code> .

B. Special cases — need more than a file upload

- 1. ACAPS Risk List — full manual review process, see [above](#). Last completed review found on disk: 2025-07-11.
- 2. FEWS NET — `fews_collect_api()` (`indicators.R`) downloads from a **hardcoded, dated URL** published by the World Bank Data Catalog (dataset `DR0091743`). Every ~2 months, get the new URL from <https://datacatalog.worldbank.org/search/dataset/0064614> and update `download_url` in the function. The function itself prints a reminder once 60 days have passed since the last known edition. It is currently **commented out** in `01-update-inputs.R` — confirm whether it should be re-enabled before relying on fresh FEWS data.
- 3. WFP Hunger Hotspots — `fao_wfp_web_collect()` scrapes [hungerhotspots.org](https://www.hungerhotspots.org/) using session tokens (`cClientSession`, `cfid`, `CF_CLIENT_*`) that expire. Refresh them via browser DevTools on the site and update the constants in `indicators.R`. This function is currently **commented out**, so `fao_wfp_web_process()` (which is active in the Food Security dimension) is serving whatever was last successfully scraped — confirm the archive date in `output/inputs-archive/fao-wfp-web/` before trusting this indicator.
- 4. Food Price Inflation — automated via `fpi_collect_api()` against the World Bank microdata API, but should still be spot-checked: run it and read the console output (`fpi_collect_api | metadata_date: ... | local_most_recent_date: ...`) to confirm it's actually pulling a newer edition and not silently skipping. If the API's response shape changes, the manual fallback (download from microdata.worldbank.org, unzip into `hosted-data/food-price-inflation/`) still works via the retired `fpi_collect()` / `fpi_process()` functions, though they'd need to be re-wired into the notebooks.
- 5. Credentials (`.access/`) — required locally and on Databricks for ACAPS, ACLED, IFES, and IRI collection to succeed at all.
- 6. GIC (Global Instances of Coups) — `gic_collect()` tries a live fetch from jonathanmpowell.com/coups/ first, but that host resets the TLS connection from the Databricks cluster's network specifically (firewall/IP-level block, confirmed reachable from elsewhere) — so it falls back to the most recently dated `.csv` in `hosted-data/gic/`. Update by downloading the current file from <https://jonathanmpowell.com/coups/> (the "List of coups by country" link) and dropping it into `hosted-data/gic/`, keeping a filename that contains a `YYYYMMDD` / `YYYY-MM-DD` date (e.g. `pt_20260829.csv`). The original source (uky.edu, hosting Powell & Thyne's file) is dead outright — its co-maintainer left for WVU in April 2026 — so jonathanmpowell.com's fallback file is now the only source, live or mirrored.

C. No longer manual (automated since earlier guidance)

- MPO — no longer requested by email; `mpo_collect()` now pulls directly from the World Bank PIP API.

D. Currently unused — don't spend time updating these

- `hosted-data/unhcr-idp/unhcr-idp.csv` — `idp_collect()` / `un_idp_process()` exist but are not called from either notebook.
- `mfr_watchlist_collect()` output — collected in Phase 1, never processed in Phase 2 (`mfr_process()` is commented out).
- REIGN — `reign_collect()` is commented out; its role is filled by `pseudo_reign_process()` (built from GIC + IFES).
- `fao_wfp_collect()` / `fao_wfp_process()` and `fews_collect_many()` / `fpi_collect_many()` — older manual-fallback versions, kept as commented-out reference code, not called.

Known Issues

- (Resolved incident, keep for reference) The GitHub token used to clone `monitor` and `hosted-data` onto the cluster expired/was revoked, causing `src/clone-git-repos-into-tmp.sh` to fail with `remote: Invalid username or token. ... fatal: Authentication failed for 'https://github.com:token@github.com:compoundrisk/hosted-data.git/'`. The script didn't check whether `git clone` actually succeeded, so it printed a misleading `"<repo> repository cloned"` and carried on with a missing (or stale) repo directory. Depending on which repo failed, this surfaced anywhere from `source('src/fns/prep.R')` failing with `"cannot open file"` to, several steps later and much more confusingly, `ghsi_collect()` failing on `cannot open file 'hosted-data/ghsi/ghsi.csv'` — because `error_delay` defaults to `FALSE` locally/interactively, that error stops the whole run. Symptom was also inconsistent between "run cell by cell" (which could reuse an already-cloned `/tmp/crm/monitor` from before the token broke) and "Run All" (which starts fresh and re-triggers the clone). **Fixed:** the token now lives in a Databricks secret (scope `compoundriskmonitor`, key `github-pat`) instead of the plaintext `.access/github-pat.txt`, read in `run-notebooks.R` via `dbutils.secrets.get()` and passed to the bash script as the `GITHUB_PAT` env var; `clone-git-repos-into-tmp.sh` now exits immediately with a clear `FATAL:` message if a clone or fetch/merge fails, instead of limping on with a missing directory; the hardcoded `bennotkin: username` was also dropped from the clone URLs (GitHub accepts `https://<token>@github.com/...` with no username). One-time setup needed on the Databricks side: `databricks secrets create-scope compoundriskmonitor`, `databricks secrets put-secret compoundriskmonitor github-pat` (paste a fresh token with read access to both repos), and confirm the cluster's identity has `READ` on that scope.
- (Resolved incident, keep for reference) The `DECPY_CompoundRiskMonitor_Johan` cluster's init script (`compoundriskmonitor_v2.sh`, running on Databricks Runtime 16.4.x — **not the same as the older `init-script/compoundriskmonitor.sh`** already in this repo) installed `r-base` / `r-base-dev` via `apt-get`. `DBR 16.4.x` already bundles its own R build with a matching IRkernel; installing `r-base` via `APT` overwrites the bundled R (`libR.so` / `/usr/bin/R`), so the IRkernel (compiled against the original bundled R) crashes on an ABI mismatch as soon as the R REPL tries to start. This surfaced as every job run failing with `ReplStartFailureException: Kernel exited while we were waiting for the kernel_info_reply message` — the notebook code (including the `.libPaths()` / `library(rlang, ...)` / `library(cli, ...)` lines at the top of the notebooks) never even ran. **Fixed** and now version-controlled at [init-script/compoundriskmonitor_v2.sh](#) — `r-base` / `r-base-dev` were removed from the `apt-get install` line (DBR already provides R and headers); everything else (the local-compile workaround for `sf` / `lwgeom` / `rgdal` / `terra` / `exactextractr` / `pdftools`, the DBFS-lock cleanup, and the `.so` dependency verification step) was preserved as-is. **The cluster's init script setting must point at this file** (or be updated to match it) for the fix to take effect — editing the file in git alone does not change what the cluster runs. If a future compile step needs headers, add back only `r-base-dev`, pinned to the exact version already bundled with the runtime (`dpkg -l r-base-core` on a fresh cluster) — never plain `r-base`.
- (Resolved incident, keep for reference) `run_type` (`01-update-inputs.R` / `02-process-indicators.R`) is read via `dbutils.widgets.get("run_type")`, but the widget was never declared anywhere in the notebooks and the "New Daily job" task had no `run_type` parameter configured — so `.get()` always threw and silently fell back to `"manual"`, **even on scheduled runs**. This meant every job run, scheduled or not, wrote to `output/manual/` instead of `output/scheduled/`, which made `output/scheduled/crm-excel/` (and everything else under `output/scheduled/`) look stale for a long time even though the pipeline was actually running fine — see the callout in [General Architecture](#) for the full explanation. **Fixed** by adding a `run_type = scheduled` parameter to the job's task in the Databricks Workflows UI (no code change was needed or possible — this can only be set from the Job configuration itself).
- **The Power BI / Excel dashboard connection is broken, and has been for a while.** In Databricks, `crm-dashboard-prod.csv` has a broken connection to the final archive that Power BI expects. This has been a genuine pain to track down, it is still not fixed, and it has already been escalated to a number of people without resolution. See [What's Next?](#) for more detail and a recommendation.
- ACAPS Risk List and Food Price Inflation were both flagged as behind on the live dashboard — verify both after the next full run rather than assuming the automation is keeping them current.
- FEWS NET collection is currently disabled (commented out); the URL it uses when re-enabled needs a manual bump every ~2 months.
- WFP Hunger Hotspots is running on a stale scrape because the session tokens it needs have not been refreshed and the collector is disabled.
- GIC (coup data) — the original uky.edu source is dead (its maintainer left for WVU in April 2026); the replacement source, jonathanmpowell.com/coups/, is unreachable from the Databricks cluster's network (TLS connection reset, not an application-level error). `gic_collect()` tries the live fetch and falls back to a manually-mirrored `hosted-data/gic/*.csv` on failure — see [Manual Maintenance Checklist](#) item B.6 for how to update it.

- **Macro Fiscal** dimension is single-source (EIU only) even though an MFR Watchlist collector still runs — worth deciding whether to wire it back in or remove the now-pointless collection call.
- As of this review, all sources in the checklist above (GHSI, GFSI, EIU, FSI, FCS, IMF Unemployment, INFORM Risk, Proteus) are confirmed up to date. Re-check `hosted-data/<source>/` for the newest filename periodically — this status will drift as new editions are published.
- A local run has **no fault tolerance** by default (`error_delay` defaults to `FALSE`): one broken source stops the whole script. Set `error_delay <- TRUE` before sourcing the notebooks if you want a best-effort run that skips failing sources.

Use Cases

Case 1: Daily Automatic Update

```
DATABRICKS JOB (daily-run-job, ~10am ET)
├ clones monitor (databricks branch), hosted-data, monitor-xlsx
├ runs run-notebooks.R
├ writes /dbfs/mnt/CompoundRiskMonitor/output/scheduled/crm-dashboard-data.csv
├ writes /dbfs/mnt/CompoundRiskMonitor/production/crm-dashboard-prod.csv
└ Dashboard reads the production file directly
```

Case 2: Local Debugging / Generating the ACAPS Review Files

```
LOCAL
├ source("src/db-notebooks/01-update-inputs.R")
├ source("src/db-notebooks/02-process-indicators.R")
├ review hosted-data/acaps-*temp-auto.csv
├ save + push reviewed files to hosted-data (databricks branch)
└ next scheduled Databricks run will pick up the review
```

Case 3: Add a New Data Source

1. Write `collect_new_indicator()` in `src/fns/indicators.R`
2. Call it in `01-update-inputs.R`, wrapped in `delay_error(..., on = error_delay)`
3. Write `new_indicator_process()` in `src/fns/indicators.R`
4. Call it inside the relevant `aggregate_dimension(...)` call in `02-process-indicators.R`
5. Add metadata to `src/indicators-list.csv`
6. Test locally (with `error_delay <- TRUE` if you expect noisy failures elsewhere)
7. Push to the databricks branch

References

- **Dashboard**: <https://compoundrisk.worldbank.org>
- **Technical Note**: <https://compoundrisk.github.io/note/>
- **Monitor repo**: <https://github.com/compoundrisk/monitor>
- **Hosted data repo**: <https://github.com/compoundrisk/hosted-data>
- **Monitor-xlsx repo**: <https://github.com/compoundrisk/monitor-xlsx>
- **ACAPS review process README**: `hosted-data/acaps-risk-list-reviewed/README.md`
- **Databricks workspace**: <https://eastus.azuredatabricks.net>
- **Databricks help**: Kartheek Kandikuppa (kkandikuppa@worldbankgroup.org)
- **Dashboard contact**: Parisa Nazari Jam (pnazarijam@worldbankgroup.org) / Anna Kojzar (akojzar@worldbankgroup.org)

What's Next?

The Power BI / Excel dashboard connection is broken, and has been for a while. In Databricks, `crm-dashboard-prod.csv` has a broken connection to the final archive that Power BI expects. This has been a genuine pain to track down, it is still not fixed, and it has already been escalated to a number of people without resolution.

The public dashboard's "Data download" link is not automatically updated. Keeping `crm.xlsx` current is a manual process: download the updated per-dimension CSVs from Databricks (`output/scheduled/crm-excel/...`, as described in [Step 2.5: Public Excel and indicator dates](#)), open `crm.xlsx`, run "Refresh All" to pull in the new data, and then manually upload the refreshed file to the `compoundrisk/monitor-xlsx` GitHub repo — `crm.xlsx`, served from <https://github.com/compoundrisk/monitor-xlsx/raw/main/crm.xlsx>, embedded on the landing page at <https://compoundrisk.worldbank.org/app-landing>.

Recommendation: rather than continuing to fight this Power BI / Excel-link setup — or even just lifting the same Excel-based dashboard onto Posit Connect as a static re-host — it would be better to build a new dashboard directly on top of the data structure that already exists (`crm-dashboard-data.csv` / `production/crm-dashboard-prod.csv`), as a **Dash or R Shiny app**, and deploy it on **Posit Connect**, the same way the other already-deployed dashboards work: hosted externally and simply embedded in the page, instead of routing through a fragile Excel/Power BI data connection. This is a good, well-scoped task to hand to the next ETC.

This document reflects a direct review of `01-update-inputs.R`, `02-process-indicators.R`, `run-notebooks.R`, `indicators.R`, `helpers.R`, `aggregation.R`, `prep.R` on the `databricks` branch, plus the current contents of a local `hosted-data` checkout, as of the date at the top of this document. Source-code line numbers and filenames change over time — if something here looks off, the code and the actual `hosted-data` folder are the source of truth.